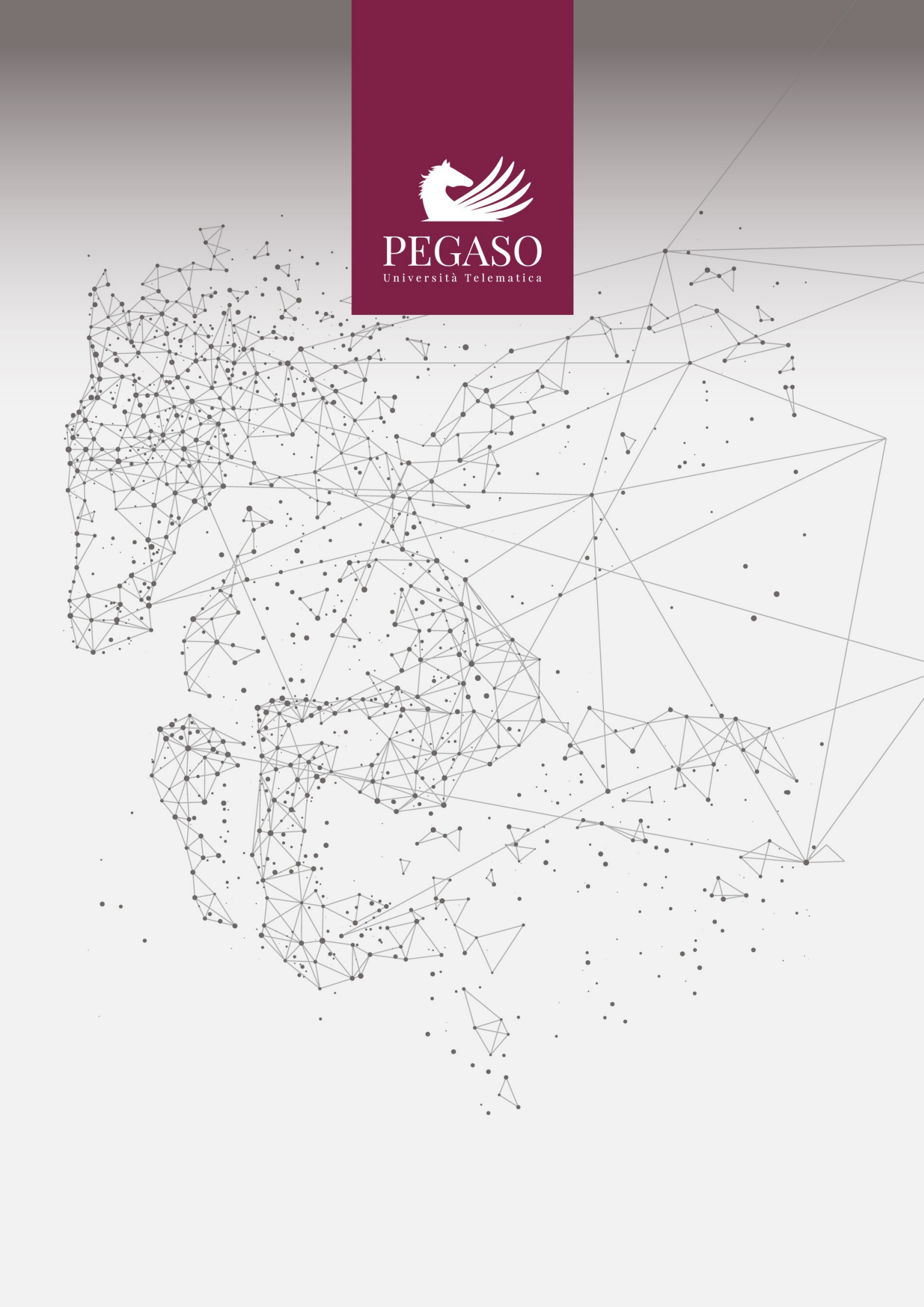

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. TRANSIENT MESSAGING CON SOCKET 3

2. TRANSIENT MESSAGING AVANZATO 6

3. MESSAGE PASSING INTERFACE 8

BIBLIOGRAFIA 11

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Transient Messaging con socket

Le chiamate di procedura remota (RPC) e le invocazioni di oggetti remoti contribuiscono all'occultamento della comunicazione nei sistemi distribuiti, ovvero migliorano la trasparenza dell'accesso. Sfortunatamente, nessuno dei due approcci è adatto in ogni occasione. Quando non si ha la certezza che il lato ricevente stia eseguendo nel momento in cui viene inviata una richiesta, sono necessari servizi di comunicazione alternativi. Analogamente, potrebbe essere necessario sostituire la natura sincrona intrinseca delle RPC, che causa il blocco di un client fino all'elaborazione della richiesta.

Questa lezione si concentra sulla comunicazione orientata ai messaggi nei sistemi distribuiti, iniziando con un esame dettagliato del comportamento sincrono e delle sue implicazioni.

Numerosi sistemi e applicazioni distribuiti sono costruiti direttamente sul semplice modello orientato ai messaggi del livello di trasporto. Al fine di comprendere e valutare meglio i sistemi orientati ai messaggi come parte delle soluzioni middleware, discuteremo in primo luogo della messaggistica tramite socket a livello di trasporto.

Particolare attenzione è stata prestata alla standardizzazione dell'interfaccia del livello di trasporto in modo che i programmatori possano utilizzare l'intera suite di protocolli (di messaggistica) con un semplice insieme di operazioni. Inoltre, le interfacce standard facilitano la portabilità di un'applicazione su una macchina diversa. Come esempio, discutiamo brevemente l'interfaccia socket introdotta nel 1970 da Berkeley Unix e adottata come standard POSIX (con pochissime modifiche).

Concettualmente, un socket è un endpoint di comunicazione in cui un'applicazione può scrivere dati da trasmettere sulla rete sottostante e da cui può leggere i dati in ingresso. Un socket è un'astrazione della porta effettiva utilizzata dal sistema operativo locale per un particolare protocollo di trasporto. Ci concentreremo in particolare sulle operazioni socket TCP illustrate nella Figura 1

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection

Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

In genere, i server eseguono le prime quattro operazioni nell'ordine specificato. Il chiamante crea un nuovo endpoint di comunicazione per un protocollo di trasporto specifico quando chiama l'operazione *socket*. Il sistema operativo locale riserva risorse per l'invio e la ricezione di messaggi per il protocollo specificato quando viene creato un endpoint di comunicazione.

L'operazione *bind* assegna al socket appena creato un indirizzo locale.

Un server, ad esempio, deve associare l'indirizzo IP della sua macchina e un numero di porta (potenzialmente noto) ad un socket. L'associazione informa il sistema operativo che il server accetterà messaggi solo sull'indirizzo e sulla porta specificati. L'indirizzo viene utilizzato per ricevere richieste di connessione in entrata in caso di comunicazione orientata alla connessione.

Solo nel caso di comunicazione basata sulla connessione viene richiamata l'operazione *listen*. Si tratta di una chiamata non bloccante che consente al sistema operativo locale di riservare buffer sufficienti per il numero massimo di richieste di connessione che il chiamante è disposto ad accettare.

L'operazione *accept* blocca il chiamante fino a quando non riceve una richiesta di connessione. Quando arriva una richiesta, il sistema operativo locale crea e restituisce un nuovo socket con le stesse proprietà dell'originale. Questo metodo consente al server, ad esempio, di eseguire un fork di un processo che gestirà quindi la comunicazione effettiva sulla nuova connessione. Il server può tornare al socket originale e attendere un'altra richiesta di connessione.

Consideriamo ora il lato client. Analogamente, è necessario creare un socket utilizzando l'operazione *socket*; tuttavia, non è necessario associare esplicitamente il socket ad un indirizzo locale perché il sistema operativo può allocare dinamicamente una porta quando viene stabilita la connessione. L'operazione *connect* richiede che il chiamante specifichi l'indirizzo a livello di trasporto a cui inviare una richiesta di connessione. Il client viene bloccato fino a quando non viene stabilita correttamente una

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

connessione, dopodiché le operazioni di invio e ricezione possono essere utilizzate per lo scambio di dati. Quando si utilizzano socket, la chiusura di una connessione è simmetrica e viene eseguita sia dal client che dal server che chiama l'operazione di chiusura. La figura 2 illustra il modello generale seguito da un client e da un server per la comunicazione orientata alla connessione tramite socket.

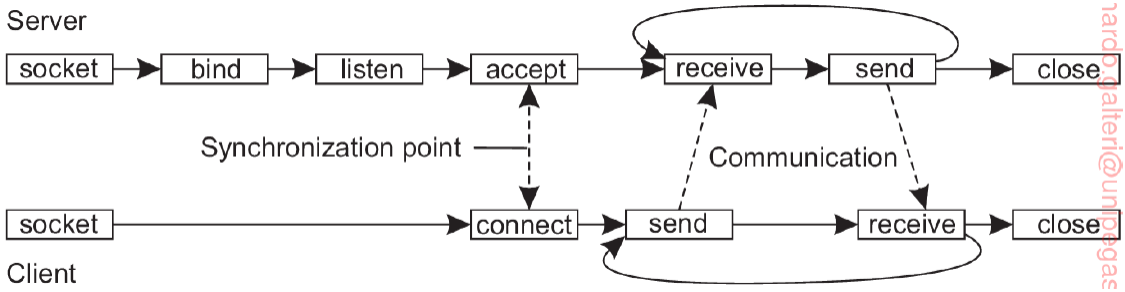


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Transient messaging avanzato

L'approccio standard basato su socket al transient messaging è fondamentale e, di conseguenza, piuttosto fragile: gli errori sono semplici da commettere. Inoltre, i socket supportano solo TCP e UDP, pertanto eventuali funzionalità di messaggistica aggiuntive devono essere implementate separatamente dallo sviluppatore dell'applicazione.

In pratica, abbiamo spesso bisogno di approcci più sofisticati per la comunicazione orientata ai messaggi al fine di semplificare la programmazione di rete, espandersi oltre le funzionalità fornite dai protocolli di rete esistenti, fare un uso migliore delle risorse locali, ecc.

Una strategia per semplificare la programmazione di rete si basa sull'osservazione che numerose applicazioni di messaggistica e i loro componenti possono essere efficacemente organizzati secondo un numero limitato di semplici modelli di comunicazione. Migliorando successivamente i socket per ciascuno di questi modelli, potrebbe essere più semplice creare un'applicazione distribuita in rete. Questa strategia è stata implementata in ZeroMQ.

ZeroMQ, così come l'approccio di Berkeley, fornisce socket attraverso i quali avviene tutta la comunicazione. La trasmissione effettiva dei messaggi avviene in genere tramite connessioni TCP e, come TCP, tutte le comunicazioni sono fondamentalmente orientate alla connessione, il che significa che deve essere stabilita una connessione tra un mittente e un destinatario prima della trasmissione del messaggio.

Tuttavia, la creazione e la manutenzione della connessione vengono gestite principalmente dietro le quinte, quindi gli sviluppatori di applicazioni non devono preoccuparsi di loro. Per semplificare ulteriormente le cose, un socket può essere associato a più indirizzi, consentendo a un server di elaborare efficacemente i messaggi provenienti da un'ampia varietà di fonti attraverso un'unica interfaccia. Un server può, ad esempio, ascoltare su più porte utilizzando una singola operazione di ricezione di blocco. I socket ZeroMQ possono quindi supportare la comunicazione molti-a-uno rispetto alla comunicazione uno-a-uno supportata dai socket Berkeley. I socket ZeroMQ supportano inoltre la comunicazione uno-a-molti, nota anche come multicasting.

La comunicazione asincrona è fondamentale per ZeroMQ: un mittente continuerà ad eseguire in genere dopo aver inviato un messaggio al sottosistema di comunicazione sottostante. Un effetto collaterale interessante della combinazione di comunicazione asincrona e orientata alla connessione è che un processo può richiedere una configurazione della connessione e quindi inviare un messaggio anche se il destinatario non è ancora pronto ad accettare richieste di connessione, e quindi anche i messaggi. Ovviamente, una richiesta di connessione e i messaggi successivi sono in coda sul lato mittente, mentre un thread separato

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

all'interno della libreria di ZeroMQ è responsabile di stabilire la connessione e trasmettere i messaggi al destinatario.

ZeroMQ semplifica la comunicazione basata su socket stabilendo un livello più elevato di astrazione mediante accoppiamento dei socket: un tipo specifico di socket utilizzato per l'invio dei messaggi è abbinato a un corrispondente tipo di socket utilizzato per la ricezione dei messaggi. Ogni coppia di tipi di socket corrisponde a uno specifico schema di comunicazione.

ZeroMQ supporta i tre modelli di comunicazione più importanti: richiesta-risposta, pubblicazione-sottoscrizione e pipeline.

Il modello richiesta-risposta viene utilizzato nelle comunicazioni client-server convenzionali, ad esempio quelle utilizzate per le chiamate di procedura remota. Utilizzando un socket di richiesta (di tipo REQ), un'applicazione client invia un messaggio di richiesta a un server e attende una risposta appropriata.

Si presuppone che il server utilizzi un socket di risposta (di tipo REP). Il modello richiesta-risposta semplifica le cose per gli sviluppatori eliminando la necessità di richiamare le operazioni di accettazione e ascolto. Inoltre, quando un server riceve un messaggio, le chiamate successive da inviare vengono automaticamente indirizzate al mittente originale. Allo stesso modo, quando un client chiama l'operazione *recv* (per ricevere un messaggio) dopo aver inviato un messaggio, ZeroMQ presuppone che il client sia in attesa di una risposta dal destinatario originale. Si noti che questa strategia è stata efficacemente codificata nell'operazione di *sendrecv* locale.

In un'architettura di pubblicazione-sottoscrizione (publish-subscribe), i client sottoscrivono particolari messaggi pubblicati dai server. In pratica, verranno trasmessi solo i messaggi a cui il client ha aderito. Questi messaggi andranno persi se un server pubblica messaggi a cui nessuno si è iscritto. Questo modello, nella sua forma più semplice, stabilisce messaggi multicast da un server a più client. Si presuppone che il server utilizzi un socket di tipo PUB, mentre i client debbano utilizzare socket di tipo SUB. Ogni socket client è connesso al socket del server. Per impostazione predefinita, un client non sottoscrive alcun messaggio specifico. Ciò indica che un client non riceverà un messaggio pubblicato dal server a meno che non venga fornita una sottoscrizione esplicita.

Infine, il modello pipeline è caratterizzato dal fatto che un processo vuole diffondere i suoi risultati nel sistema, supponendo che altri processi vogliano estrarre quei risultati. L'essenza del modello di pipeline è che un processo di push non si preoccupa di quale altro processo estrae i suoi risultati; il primo che è disponibile è sufficiente. Allo stesso modo, qualsiasi processo che estrae risultati da più altri processi lo farà dal processo di push iniziale che rende disponibili i suoi risultati. Pertanto, l'intento del modello pipeline è quello di mantenere attivi il maggior numero possibile di processi, accelerando il flusso dei risultati attraverso una pipeline di processi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Message Passing Interface

Con l'introduzione di computer ad alte prestazioni, i programmatori hanno cercato operazioni orientate ai messaggi che facilitassero la creazione di applicazioni altamente efficienti. Ciò indica che le operazioni devono essere a un livello di astrazione che facilita lo sviluppo di applicazioni e che la loro implementazione deve comportare un carico minimo. I socket sono stati ritenuti inadeguati per due motivi. In primo luogo, erano al livello errato di astrazione perché supportavano solo le operazioni di invio e ricezione di base. I socket sono stati progettati per comunicare tra reti utilizzando stack di protocollo generici come TCP/IP. Sono stati ritenuti incompatibili con i protocolli proprietari progettati per le reti di interconnessione ad alta velocità, come quelli utilizzati nei cluster di server ad alte prestazioni. Questi protocolli richiedevano un'interfaccia in grado di gestire funzionalità avanzate, come varie tecniche di buffering e sincronizzazione.

Di conseguenza, le librerie di comunicazione proprietarie sono state fornite con la maggior parte delle reti di interconnessione e dei computer ad alte prestazioni. Queste librerie fornivano un'ampia gamma di operazioni di comunicazione avanzate e generalmente efficaci. Ovviamente, tutte le librerie erano incompatibili tra di loro, creavano quindi un problema di portabilità per gli sviluppatori di applicazioni.

Alla fine, la necessità di indipendenza dall'hardware e dalla piattaforma ha portato alla creazione di uno standard per la trasmissione dei messaggi noto come Message Passing Interface o MPI. Poiché MPI è progettato per applicazioni parallele, è ottimizzato per la comunicazione transitoria ed utilizza direttamente la rete sottostante. Inoltre, presuppone che errori irreversibili come arresti anomali del processo e partizioni di rete non richiedano il ripristino automatico.

MPI presuppone che la comunicazione avvenga all'interno di un gruppo di processi noti. Ogni gruppo riceve un identificatore univoco. Inoltre, a ogni processo all'interno di un gruppo viene assegnato un identificatore locale. Una coppia (groupID, processID) viene quindi utilizzata al posto di un indirizzo a livello di trasporto per identificare in modo univoco l'origine o la destinazione di un messaggio. Possono esserci più gruppi potenzialmente sovrapposti di processi in esecuzione simultanea coinvolti in un calcolo.

Nella Figura 3 sono riepilogate le operazioni di messaggistica più intuitive per supportare la comunicazione temporanea che sono alla base di MPI.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Tramite l'operazione MPI_*bsend* è supportata la comunicazione asincrona transitoria. In genere, il mittente invia un messaggio per la trasmissione, che viene quindi copiato in un buffer locale nel sistema di runtime MPI. Una volta creata una copia del messaggio, il mittente continua ad eseguire. Non appena un destinatario chiama un'operazione di ricezione, il sistema di runtime MPI locale rimuoverà il messaggio dal buffer locale e gestirà la trasmissione.

Esiste anche un'operazione di invio bloccante denominata *MPI_send* con semantica dipendente dall'implementazione. L'operazione *MPI_send* può bloccare il chiamante fino a quando il messaggio specificato non è stato copiato nel sistema di runtime MPI del mittente o il destinatario non ha avviato un'operazione di ricezione. L'operazione *MPI_send* consente la comunicazione sincrona in cui il mittente si blocca fino a quando la sua richiesta non viene accettata per un'ulteriore elaborazione. Ultimo ma non meno importante, è supportata anche la forma più forte di comunicazione sincrona: quando un mittente chiama *MPI_sendrecv*, invia una richiesta al destinatario e attende una risposta. Questa operazione corrisponde essenzialmente a un RPC standard.

Sia *MPI_send* che *MPI_ssend* hanno varianti che evitano di copiare i messaggi dai buffer utente ai buffer di sistema runtime MPI. Queste varianti corrispondono essenzialmente alla comunicazione asincrona. Con *MPI_isend*, un mittente passa un puntatore di messaggio al sistema di runtime MPI, che quindi gestisce la comunicazione. Il mittente continua ad eseguire immediatamente. Per evitare che il messaggio venga sovrascritto prima del completamento della comunicazione, MPI fornisce operazioni per verificarne il completamento o, se necessario, per bloccarlo.

Come per *MPI_send*, non è specificato se il messaggio è stato trasferito al ricevitore o semplicemente copiato dal sistema runtime MPI locale in un buffer interno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Analogamente, un mittente passa solo un puntatore al sistema di runtime MPI quando si utilizza *MPI_issend*. Quando il sistema di runtime indica che il messaggio è stato elaborato, il mittente è certo che il destinatario ha accettato e sta attualmente elaborando il messaggio.

Per ricevere un messaggio, viene richiamata l'operazione *MPI_recv* che blocca il chiamante fino all'arrivo del messaggio. Esiste anche una variante asincrona chiamata *MPI_irecv* che indica che un destinatario è pronto ad accettare un messaggio. Il destinatario può determinare se un messaggio è arrivato o bloccarlo fino a quando non lo fa.

La semantica delle operazioni di comunicazione MPI non è sempre semplice e talvolta è possibile sostituire operazioni diverse senza influire sulla correttezza di un programma. La ragione ufficiale per supportare così tante forme di comunicazione è quella di fornire agli sviluppatori di sistemi MPI ampie opportunità per ottimizzare le prestazioni. Giunto alla sua terza versione, MPI supporta più di 440 operazioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Stevens W. UNIX Network Programming – Networking APIs: Sockets and XTI. Prentice Hall, 2nd edition, 1998.
- Hintjens P., ZeroMQ, 2013.
- Akgul F., ZeroMQ, 2013
- Gropp W., Lusk E., and Skjellum A. Using MPI-2, Portable Parallel Programming with the Message-Passing Interface. 3rd edition, 2016.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).